

Architecture — Application de gestion de CRA

Document généré depuis le dépôt le 24 août 2026. Chaque affirmation est tirée du code, des notes de conception ou d'une exécution réelle des tests ; ce qui est conçu mais pas encore construit est marqué **planifié**.

1. Le projet en une page

Application de gestion des **comptes rendus d'activité** (CRA). Un consultant déclare son activité mensuelle jour par jour — missions et absences — puis soumet son mois ; son manager le valide ou le refuse avec un motif.

Deux rôles, portés par l'utilisateur connecté et appliqués côté serveur :

Rôle	Ce qu'il fait
Consultant	saisit son activité et ses absences, soumet son CRA
Manager	gère les missions, valide ou refuse les CRA de son équipe

Le projet est une **démonstration** : il montre comment une application se construit avec des assistants IA en 2026. L'authentification est volontairement bouchonnée — choisir un profil tient lieu de connexion — afin de concentrer l'effort sur le métier.

Le périmètre fonctionnel est découpé en **17 user stories unitaires**, réalisées dans l'ordre de leur numéro, chacune ne dépendant que des précédentes.

2. Stack et structure

Versions réellement installées :

Côté	Technologies
Frontend	React 19.2, TypeScript 6.0 (strict), Vite 8.2, React Router 7.18, Vitest 4.1
Backend	Python 3.12, FastAPI 0.141, SQLAlchemy 2.0, Pydantic 2.13, Uvicorn 0.52
Base de données	SQLite, fichier <code>backend/cra.db</code> , alimentée au démarrage
Qualité	pytest 9.1 + coverage, Ruff 0.16, Vitest + Testing Library, oxlint

```
sample/
├─ backend/app/      main.py · core/ · db/ · models/ · schemas/ · services/ · routers/
├─ backend/tests/    api/ · services/ · core/ · schemas/
├─ frontend/src/     api/ · types/ · hooks/ · components/ · pages/ · styles/
├─ docs/             architecture/ · adr/
└─ specs/            les 17 user stories, en français
```

Les dépendances sont ajoutées **quand une story en a besoin**, jamais par anticipation.

3. Vue d'ensemble

Deux exécutables distincts, aucune dépendance de build entre eux :

```

Navigateur → Vite (5173) —/api→ FastAPI (8000) → SQLite (cra.db)
           |
           └ sert le SPA React en développement

```

Tout passe par le port **5173** : le proxy Vite renvoie `/api` vers le port 8000. Tester directement le port 8000 contourne le proxy, l'en-tête d'identité et donc une bonne part de ce qu'il faut vérifier. `CORSMiddleware` reste configuré comme filet de sécurité.

L'identité de l'utilisateur courant voyage dans l'en-tête `X-Demo-User`, posé une seule fois par le client HTTP partagé.

4. Modèle de domaine

Entités construites :

Entité	Rôle	Champs structurants
User	consultant ou manager	name, email, role, manager_id
Mission	engagement client	name, client, start_date, end_date, status
Assignment	consultant ↔ mission	user_id, mission_id

Entités **planifiées** (US-007 et suivantes) : `Cra` (un mois, un statut) et `CraEntry` (une déclaration sur un jour).

Énumérations définies dans `models/enums.py` : `UserRole`, `MissionStatus`, `CraStatus`, `EntryType`. Les deux dernières sont déjà posées bien que le CRA ne soit pas encore construit : elles fixent le vocabulaire du domaine.

Invariants et lieu où ils sont appliqués :

Invariant	Appliqué dans
Nom de mission unique par client	contrainte de base + service
Une affectation par couple (mission, consultant)	contrainte de base
Seul un manager crée, modifie, clôture une mission	dépendance FastAPI, côté serveur
Somme des fractions d'un jour ≤ 1	<i>planifié</i> — service CRA

Les valeurs dérivées — totaux mensuels, compteurs — sont **calculées à la lecture**, jamais stockées : une valeur dérivée persistée finit toujours par diverger de sa source.

5. Contrat d'API

Toutes les routes sont préfixées par `/api`. Le corps des réponses est en `camelCase`.

Route	Verbe	Statut	Rôle requis
/api/health	GET	200	aucun
/api/users	GET	200	aucun (profils de démonstration)
/api/missions	GET	200	authentifié
/api/missions	POST	201	Manager
/api/missions/disponibles	GET	200	Consultant
/api/missions/{id}	GET	200	authentifié
/api/missions/{id}	PUT	200	Manager
/api/missions/{id}/affectations	POST	201	Manager
/api/missions/{id}/affectations/{userId}	DELETE	204	Manager
/api/missions/{id}/cloture	POST	200	Manager

Toute réponse non-2xx porte `{"detail": "<phrase française>"}`, y compris les 500 et les 422 — ces derniers sont aplatis en une phrase, alors que FastAPI produit par défaut une liste d'objets. Le frontend affiche ce `detail` tel quel.

6. Frontend

Découpage en couches, du plus bas au plus haut :

types/dto.ts	types miroirs des schémas backend, écrits à la main
api/client.ts	fetch partagé : en-tête X-Demo-User, 204, parsing de {detail}
api/*.ts	une fonction par endpoint
hooks/*.ts	une donnée par hook : data, status, error, reload
components/	présentation pure, aucun appel réseau
pages/	composition de l'écran et gestion des trois états

Écrans en place : accueil (état de l'API), missions, page 404. Le sélecteur de profil vit dans l'en-tête — choisir un profil **est** la connexion.

Trois états sont obligatoires sur chaque appel distant : chargement, erreur, vide. Un tableau vide sans message se lit comme une panne pendant une démonstration.

Tous les libellés visibles sont en français, les identifiants de code en anglais. Aucune couleur n'est écrite en dur : elles viennent des jetons de `styles/tokens.css`, et un statut n'est jamais signalé par la couleur seule.

7. Décisions d'architecture

ADR	Titre	Statut	Décision
0001	Monorepo à deux dossiers	Accepté	<code>backend/</code> et <code>frontend/</code> autonomes, chacun son gestionnaire de paquets et ses tests ; le périmètre d'un agent correspond exactement à un dossier
0002	Contrat d'abord sur l'OpenAPI	Accepté	Le schéma OpenAPI est la frontière ; les types TypeScript sont écrits à la main, sans génération, pour que le frontend démarre sans backend qui tourne

ADR-0002 tranche aussi le conflit `snake_case` / `camelCase` : Python reste en `snake_case`, le fil est en `camelCase`, via une classe de base `CamelModel` dont héritent tous les schémas. Ce choix devait être fait au moment du socle — le rattraper après plusieurs stories aurait été une reprise longue et risquée.

8. Qualité

Chiffres relevés le 24 août 2026, par exécution réelle :

Côté	Tests	Couverture
Backend	104 verts	98,05 %
Frontend	106 verts	92,01 % de lignes, 86,63 % de branches

Les garde-fous ne sont pas déclaratifs, ils sont outillés :

- `--cov-fail-under=70` dans `pyproject.toml` : la suite backend échoue sous 70 %.
- Seuils Vitest à 70 % sur les quatre métriques.
- `typescript/no-explicit-any` et `ban-ts-comment` en erreur dans `oxlint`.
- Ruff en `check` et `format --check`.

Les tests d'API sont écrits **avant** l'implémentation, un par critère d'acceptation, et les règles métier sont testées directement sur les services, sans passer par HTTP.

9. Ce qui reste à construire

Sur les 17 user stories, la gestion des missions et la sélection de profil sont en place. Restent à construire :

Stories	Sujet
US-007 → US-011	calendrier mensuel, saisie journée et demi-journées, absences, récapitulatif
US-012 → US-013	soumission du CRA et annulation de la soumission
US-014 → US-016	consultation des CRA en attente, validation, refus motivé
US-017	tableau de bord du consultant

Ces stories introduiront les entités `Cra` et `CraEntry`, le routeur `cra`, le routeur `validation`, le calcul des jours fériés français et le composant calendrier — tous prévus dans la note de conception du socle, sans déplacement

de dossier.